DDL (Data Definition Language) Statements:

It is a subset of SQL (Structured Query Language) used to define and manage the structure of a database. DDL commands are responsible for creating, modifying, and deleting database objects such as tables, indexes, views, and stored procedures. Used to create database tables from relational schema.

Booking

```sql
CREATE TABLE booking (
  booking_id INT AUTO_INCREMENT PRIMARY KEY,
  guest_id INT NOT NULL,
  staff_id INT,
  number_guests INT NOT NULL CHECK (number_guests > 0),
  stay_start DATE NOT NULL,
  stay_end DATE NOT NULL,
  FOREIGN KEY (guest_id) REFERENCES guest(guest_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id)
    ON DELETE SET NULL
    ON UPDATE CASCADE
);
```

Guest

```sql
CREATE TABLE guest (
  guest_id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  gender VARCHAR(10) NOT NULL,
  email VARCHAR(50) NOT NULL,
  phone_number VARCHAR(20) NULL
);
```

Staff

```sql
CREATE TABLE staff (
  staff_id INT AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  gender VARCHAR(10) NOT NULL,
  salary DECIMAL(10,2) NOT NULL CHECK (salary > 0),
  email VARCHAR(50) NOT NULL,
  phone_number VARCHAR(20),
  manager_id INT,
  FOREIGN KEY (manager_id) REFERENCES staff(staff_id)
    ON DELETE SET NULL
    ON UPDATE CASCADE

);
```

Payment

```sql
CREATE TABLE payment (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  booking_id INT NOT NULL UNIQUE,
  payment_amount DECIMAL(10,2) NOT NULL,
  method_name VARCHAR(30),
  FOREIGN KEY (booking_id) REFERENCES booking(booking_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Room

```sql
CREATE TABLE room (
  room_id INT PRIMARY KEY,
  floor_number INT,
  is_booked TINYINT(1) NOT NULL,
  type_id VARCHAR(5),
  FOREIGN KEY (type_id) REFERENCES room_type(type_id)
);
```

Room Type

```sql
Create TABLE room_type (
  type_id VARCHAR(5) PRIMARY KEY,
  type_name VARCHAR(15) NOT NULL,
  price DECIMAL(10,2) NOT NULL CHECK (price > 0),
  capacity INT NOT NULL CHECK (capacity >= 1)
);
```

Booking Room

```sql
CREATE TABLE booking_room (
  booking_id INT NOT NULL,
  room_id INT NOT NULL,
  PRIMARY KEY (booking_id, room_id),
  FOREIGN KEY (booking_id) REFERENCES booking(booking_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  FOREIGN KEY (room_id) REFERENCES room(room_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

DML (Data Manipulation Language) Statements:

Database Management System offers a framework of functions to modify or alter the data, called DML. Data manipulation includes introducing data into tables, updating the table's data, and deleting the data from the table.

Booking

```sql
INSERT INTO booking (booking_id, guest_id, staff_id, number_guests, stay_start, stay_end) VALUES
(1, 1, 2, 1, '2025-05-10', '2025-05-12'),
(2, 2, 1, 2, '2025-06-01', '2025-06-05'),
(3, 3, 3, 3, '2025-07-15', '2025-07-20'),
(4, 4, 1, 1, '2025-02-01', '2025-02-03');
```

Guest

```sql
INSERT INTO guest (guest_id, first_name, last_name, gender, email, phone_number) VALUES
(1, 'Mona', 'Hassan', 'Female', 'monahassan@gmail.com', '01012345678'),
(2, 'Ahmed', 'Khaled', 'Male', 'ahmedkhaled@gmail.com', '01087654321'),
(3, 'Sara', 'Ali', 'Female', 'saraali@gmail.com', NULL),
(4, 'Eman', 'Ebrahim', 'Male', 'EmanEbrahim@gmail.com', '01123456789');
```

Staff

```sql
INSERT INTO staff (staff_id, first_name, last_name, gender, salary, email, phone_number, manager_id) VALUES
(1, 'Omar', 'Samir', 'Male', 12000.00, 'omarsamir@gmail.com', '01110986601', NULL),
(2, 'Laila', 'Mostafa', 'Female', 15000.00, 'lailamostafa@gmail.com', '01278652208', 1),
(3, 'Hadi', 'Youssef', 'Male', 8000.00, 'hadiyoussef@gmail.com', '01098763570', 1);
```

Payment

```sql
INSERT INTO payment (payment_id, booking_id, payment_amount, method_name) VALUES
(1, 1, 500.00, 'Credit Card'),
(2, 2, 900.00, 'Cash'),
(3, 3, 1800.00, 'Debit Card');
```

Room

```sql
INSERT INTO room (room_id, floor_number, is_booked, type_id) VALUES
(101, 1, 1, 'SL'),
(104, 1, 1, 'SL'),
(202, 2, 1, 'DL'),
(205, 2, 0, 'DL'),
(303, 3, 1, 'SU'),
(406, 4, 0, 'FL');
```

Room Type

```sql
INSERT INTO room_type (type_id, type_name, price, capacity) VALUES
('DL', 'Double', 900.00, 2),
('FL', 'Family', 1800.00, 5),
('SL', 'Single', 500.00, 1),
('SU', 'Suite', 1500.00, 4);
```

Booking Room

```sql
INSERT INTO booking_room (booking_id, room_id) VALUES
(1, 101),
(2, 202),
(3, 303),
(4, 104);
```

Queries and Transaction Pathway:

1. Retrieve payments and booking details

```sql
SELECT payment.payment_id,payment.payment_amount,payment.method_name,
booking.stay_start,booking.stay_end
FROM payment
INNER JOIN booking
ON payment.booking_id=booking.booking_id;
```

payment_id	payment_amount	method_name	stay_start	stay_end
1	500.00	Credit Card	2025-05-10	2025-05-12
2	900.00	Cash	2025-06-01	2025-06-05
3	1800.00	Debit Card	2025-07-15	2025-07-20

SQL Query Logical Execution Order:

The real logical order is:

1. **FROM**
2. **JOIN / ON**
3. **WHERE**
4. **GROUP BY**
5. **HAVING**
6. **SELECT**
7. **ORDER BY**
8. **LIMIT**

So, in this **query**, SQL mainly uses:

1. **FROM payment**
2. **JOIN booking ON payment.booking_id = booking.booking_id**
3. **SELECT**

Transaction Pathway for previous Query:

Step 1: Start with the payment table (FROM)

SQL begins by reading the first table in the FROM clause.

It loads rows from **payment**, including these columns:

- payment_id
- payment_amount
- method_name
- booking_id *(needed for matching)*

Step 2: Join to the booking table (INNER JOIN)

For each row in **payment**, SQL looks for a matching row in **booking**.

Join condition:

payment.booking_id = booking.booking_id

So, SQL matches:

- each payment → the booking it belongs to

Only rows where the booking_id exists in *both* tables are kept.

Step 3: Read required columns from booking

From the matching booking row, SQL retrieves:

- stay_start
- stay_end

Step 4: Combine matching rows into one result row

SQL merges columns from both tables into a single output row:

- payment table gives → payment_id, payment_amount, method_name
- booking table gives → stay_start, stay_end

This forms the final dataset.

2. Retrieve all bookings of guests if available

```sql
SELECT guest.first_name AS
guest_name,guest.email,booking.booking_id,booking.stay_start
FROM guest
LEFT JOIN booking
ON guest.guest_id=booking.guest_id
ORDER BY guest.guest_id;
```

guest_name	email	booking_id	stay_start
Mona	monahassan@gmail.com	1	2025-05-10
Ahmed	ahmedkhaled@gmail.com	2	2025-06-01
Sara	saraali@gmail.com	3	2025-07-15
Ahmed	ahmed.khaled@gmail.com	4	2025-02-01

Step 1: Start with the guest table (FROM)

SQL begins by scanning the guest table.

For each guest row, it loads:

- first_name
- email
- guest_id *(needed for the join)*

Step 2: Perform a LEFT JOIN to the booking table

For every guest record, SQL checks if there is a matching booking:

Join condition:

guest.guest_id = booking.guest_id

- If **a match exists** → SQL links the booking row to the guest.
- If **no booking exists** → SQL still keeps the guest (because it's LEFT JOIN) and puts **NULL** in booking columns.

In the output, all guests have bookings.

Step 3: Read required columns from booking table

From the matching row in **booking**, SQL retrieves:

- booking_id
- stay_start

Step 4: Combine matched rows into the result

SQL merges:

From **guest**:

- guest_name (first_name)
- email

From **booking**:

- booking_id
- stay_start

Step 5: Order the results

Sort by:

- ORDER BY guest.guest_id

3. Retrieve all room types and their status

```sql
SELECT room_type.type_name,room_type.price,room.room_id,room.is_booked
FROM room
RIGHT JOIN room_type
ON room.type_id=room_type.type_id;
```

type_name	price	room_id	is_booked
Double	900.00	202	1
Double	900.00	205	0
Family	1800.00	406	0
Single	500.00	101	1
Single	500.00	104	1
Suite	1500.00	303	1

Step 1: Start with the room table (FROM)

SQL begins by loading rows from the **room** table.

Columns read from room:

- room_id
- type_id
- is_booked

Even though the SELECT doesn't show type_id, SQL still needs it for join.

Step 2: Apply RIGHT JOIN to room_type

Join condition:

room.type_id = room_type.type_id

Because this is a **RIGHT JOIN**:

- SQL keeps **all rows** from **room_type**
- SQL only includes matching rows from **room**

- If a room_type has *no matching rooms*, it still appears (room columns become NULL)

(Output shows one row per room that matches relevant all room types.)

Step 3: Read data from room_type table

SQL retrieves from room_type:

- type_name
- price
- type_id *(for join logic)*

Step 4: Match rows based on type_id

For each room_type:

- SQL looks for rooms with the same type_id
- If found → combine room + room_type data
- If not found → room fields become NULL (because RIGHT JOIN keeps room_type rows)

Step 5: Produce the Combined Output

SQL creates final rows using:

From **room_type**:

- type_name
- price

From **room**:

- room_id
- is_booked

4. Calculate total salaries and their average by gender

```sql
SELECT gender,
SUM(salary) AS total_salary_paid,
AVG(salary) AS average_salary
FROM staff
GROUP BY gender;
```

gender	total_salary_paid	average_salary
Female	15000.00	15000.000000
Male	20000.00	10000.000000

Step 1: Start with the staff table (FROM)

SQL scans all rows of the **staff** table.

It collects these columns for processing:

- gender
- salary

SQL loads all staff rows into memory for grouping.

Step 2: Group the rows by gender (GROUP BY)

SQL splits staff into **separate groups** based on gender:

Example grouping:

- **Female Group** → All rows where gender = 'Female'
- **Male Group** → All rows where gender = 'Male'

Each group becomes its own "bucket" for calculations.

Step 3: Calculate aggregate functions for each group

For each gender group:

SQL computes:

- **SUM(salary)** → total salaries for that gender
- **AVG(salary)** → average salary for that gender

Example from output:

Female

- SUM = 15000
- AVG = 15000
  (Only 1 female staff exists)

Male

- SUM = 20000
- AVG = 10000
  (2 male staff exist → average is 20000 ÷ 2)

Step 4: Build result rows (SELECT)

For each gender group, SQL produces one final row containing:

- gender
- total_salary_paid
- average_salary

5. Retrieve employees' name and their manager

```sql
SELECT
    s.first_name AS staff_name,
    s.last_name AS staff_last_name,
    m.first_name AS manager_name,
    m.last_name AS manager_last_name
FROM
    staff s
LEFT JOIN
    staff m ON s.manager_id = m.staff_id;
```

staff_name	staff_last_name	manager_name	manager_last_name
Omar	Samir	*NULL*	*NULL*
Laila	Mostafa	Omar	Samir
Hadi	Youssef	Omar	Samir

Step 1: Start with the staff table (s)

SQL scans all rows of the **staff** table (aliased as s).

Columns needed:

- first_name → staff_name
- last_name → staff_last_name
- manager_id → for joining to managers

Step 2: LEFT JOIN staff table again (m)

- SQL joins each staff member (s) to their manager (m) using:

s.manager_id = m.staff_id

- LEFT JOIN ensures **all staff rows are kept**, even if they don't have a manager.
- If a staff member has no manager → manager_name and manager_last_name = NULL.

Step 3: Retrieve manager columns from m

- m.first_name → manager_name
- m.last_name → manager_last_name

SQL now has all required columns for each staff row.

Step 4: Build final result (SELECT)

For each staff row, SQL produces:

| staff_name | staff_last _name | manager_name | manager_last_ name |

6. Retrieve full booking details

```sql
SELECT guest.first_name AS guest_name,
room.room_id,room_type.type_name,room_type.price,
booking.stay_start,booking.stay_end
FROM guest
INNER JOIN booking ON guest.guest_id=booking.guest_id
INNER JOIN booking_room ON booking.booking_id=booking_room.booking_id
INNER JOIN room ON booking_room.room_id=room.room_id
INNER JOIN room_type ON room.type_id=room_type.type_id;
```

guest_name	room_id	type_name	price	stay_start	stay_end
Ahmed	202	Double	900.00	2025-06-01	2025-06-05
Mona	101	Single	500.00	2025-05-10	2025-05-12
Ahmed	104	Single	500.00	2025-02-01	2025-02-03
Sara	303	Suite	1500.00	2025-07-15	2025-07-20

Step 1: Start with guest table

- SQL reads all guest rows.
- Columns used: **guest_id, first_name**

Step 2: INNER JOIN booking

- SQL matches each guest to their bookings using:

  guest.guest_id = booking.guest_id

- Only guests with bookings are included.
- Columns retrieved from booking: booking _id, stay _start, stay _end

---

Step 3: INNER JOIN booking_room

SQL links each booking to the rooms booked:

Booking.booking_id = booking_room .booking_id

Retrieves **room_id** from **booking_room**

Step 4: INNER JOIN room

SQL finds room details for each booked room:

booking_room.room_id = room.room_id

Columns retrieved from room: room_id, type_id

Step 5: INNER JOIN room_type

SQL retrieves room type details for each room:

room.type_id = room_type.type_id

Columns retrieved: type_name, price

Step 6: Build final output (SELECT)

Selects only the required columns:

guest.first_name AS guest_name

- room.room_id
- room_type.type_name
- room_type.price
- booking.stay_start
- booking.stay_end

7. Retrieve available double room

```sql
SELECT room.room_id,room.floor_number
FROM room
INNER JOIN room_type ON room.type_id=room_type.type_id
WHERE room_type.type_name='Double' AND room.is_booked=0;
```

room_id	floor_number
205	2

Step 1: Start with the room table

- SQL scans all rows of the room table.
- Columns retrieved: room_id, type_id, floor_number, is_booked

---

Step 2: INNER JOIN room_type

- SQL joins each room to its room type using:

room.type_id = room_type.type_id

- Only rooms with a matching type_id in room_type are included.
- Columns retrieved from room_type: type_name

Step 3: Apply the WHERE filter

SQL filters rows where:

1. room_type.type_name = 'Double' → only rooms of type Double
2. room.is_booked = 0 → only rooms that are currently not booked

---

Step 4: Build final output (SELECT)

Columns retrieved for the final output

8. Calculate the stay duration for each guest

```sql
SELECT guest.first_name,guest.last_name,booking.stay_start,booking.stay_end,
DATEDIFF (booking.stay_end, booking.stay_start) AS duration_in_days
FROM guest
INNER JOIN booking ON guest.guest_id=booking.guest_id;
```

first_name	last_name	stay_start	stay_end	duration_in_days
Mona	Hassan	2025-05-10	2025-05-12	2
Ahmed	Khaled	2025-06-01	2025-06-05	4
Sara	Ali	2025-07-15	2025-07-20	5
Ahmed	Khaled	2025-02-01	2025-02-03	2

Step 1: Start with the guest table

- SQL scans all rows of the **guest** table.
- Columns retrieved: guest_id, first_name, last_name

Step 2: INNER JOIN booking

- SQL matches each guest with their bookings using:

  guest.guest_id = booking.guest_id
- Only guests **with bookings** are included.
- Columns retrieved from booking: stay_start, stay_end

Step 3: Calculate duration_in_days

- SQL computes the difference between stay_end and stay_start for each row:

DATEDIFF (booking.stay_end, booking.stay_start) AS duration_in_days

- This gives the total stay duration in days for each booking.

Step 4: Build final output (SELECT)

9. Retrieve staff member with a high number of bookings

```sql
SELECT
    staff.first_name,
    staff.last_name,
    COUNT(booking.booking_id) AS total_bookings_handled
FROM
    staff
INNER JOIN
    booking ON staff.staff_id = booking.staff_id
GROUP BY
    staff.staff_id, staff.first_name, staff.last_name
HAVING
    COUNT(booking.booking_id) > 1;
```

first_name	last_name	total_bookings_handled
Omar	Samir	2

Step 1: Start with the staff table

- SQL reads all rows of the **staff** table.
- Columns used: staff_id, first_name, last_name

Step 2: INNER JOIN booking

- SQL matches each staff member to the bookings they handle:

  staff.staff_id = booking.staff_id
- Only staff with at least one booking are included.
- Columns retrieved from booking: booking_id

Step 3: Group rows by staff member (GROUP BY)

- SQL groups rows for each staff member based on:

staff_id, first_name, last_name

- Each group now represents **all bookings handled by one staff member**.

Step 4: Compute aggregates (SELECT)

- For each staff group, SQL calculates:

  COUNT (booking.booking_id) AS total_bookings_handled

- This gives the number of bookings each staff member handled.

Step 5: Apply HAVING filter

- SQL keeps only staff with:

  total_bookings_handled > 1

- Staff with 1 or 0 bookings are excluded.

Step 6: Build final output (SELECT)

10. View of financial booking status

```sql
CREATE VIEW v_financial_booking_status AS
SELECT
    guest.first_name AS guest_name,
    booking.booking_id,
    booking.stay_start,
    booking.stay_end,
    DATEDIFF(booking.stay_end, booking.stay_start) AS duration_in_days,
    payment.payment_amount,
    payment.method_name,
    room_type.type_name,
    room_type.price AS daily_room_price
FROM
    guest
INNER JOIN booking  ON guest.guest_id= booking.guest_id
INNER JOIN booking_room  ON booking.booking_id= booking_room.booking_id
INNER JOIN room  ON booking_room.room_id = room.room_id
INNER JOIN room_type  ON room.type_id = room_type.type_id
INNER JOIN payment ON booking.booking_id=payment.booking_id;

```

SQL Query Logical Execution Order:

1. **FROM guest** → start with the guest table
2. **INNER JOIN booking ON guest.guest_id = booking.guest_id** → link each guest to their bookings
3. **INNER JOIN booking_room ON booking.booking_id = booking_room.booking_id** → link each booking to its rooms
4. **INNER JOIN room ON booking_room.room_id = room.room_id** → get room details
5. **INNER JOIN room_type ON room.type_id = room_type.type_id** → get room type and daily price
6. **INNER JOIN payment ON booking.booking_id = payment.booking_id** → get payment information
7. **WHERE** (not used)

8. **SELECT** → choose the output columns and calculate DATEDIFF
9. **CREATE VIEW** → store the result as v_financial_booking_status

---

Transaction Pathway for previous Query:

Step 1: Start with the guest table

- SQL scans all guests.
- Columns used: guest_id, first_name

Step 2: INNER JOIN booking

- SQL matches each guest to their bookings:

  guest.guest_id = booking.guest_id
- Only guests with bookings are included.
- Columns retrieved: booking_id, stay_start, stay_end

Step 3: INNER JOIN booking_room

- SQL links each booking to its booked rooms:

  booking.booking_id = booking_room.booking_id
- Column retrieved: room_id

Step 4: INNER JOIN room

- SQL retrieves room details:

  booking_room.room_id = room.room_id
- Columns retrieved: room_id, type_id

Step 5: INNER JOIN room_type

- SQL retrieves room type details:

  room.type_id = room_type.type_id
- Columns retrieved: type_name, price (daily room price)

Step 6: INNER JOIN payment

- SQL links booking to its payment(s):

  booking.booking_id = payment.booking_id

- Columns retrieved: payment_amount, method_name

Step 7: Calculate duration_in_days

- SQL computes:

DATEDIFF (booking.stay_end, booking.stay_start) AS duration_in_days

- Represents total stay duration for each booking.

Step 8: Build final output (SELECT)

```
1 SELECT * FROM v_financial_booking_status WHERE payment_amount > 1000;
```

guest_name	booking_id	stay_start	stay_end	duration_in_days	payment_amount	method_name	type_name	daily_room_price
Sara	3	2025-07-15	2025-07-20	5	1800.00	Debit Card	Suite	1500.00

What it does

1. FROM v_financial_booking_status

- The query uses the **VIEW** created earlier, which already combines guest info, booking details, room info, and payment data.

2. WHERE payment_amount > 1000

- Filters the rows to include only **bookings where the guest paid more than 1000**.
- This could include deposits, partial payments, or full payments depending on the data.

3. SELECT *

- Returns **all columns** from the VIEW for those bookings.

Resulting Insight

- You will see only the bookings with **high payments** (greater than 1000).
- Useful for:
  - Quickly identifying **high-value bookings**
  - Monitoring **large deposits** or **high-value payments**
  - Financial reporting or VIP guest tracking

In the previous query

- The **guest Sara paid 1500.00** — this could be a **deposit** or a **partial payment**.
- The **full booking cost** is higher, calculated as:

Daily room price × duration_in_days = 1500 × 5 = 7500.00

- So, the **payment_amount (1500.00)** is **less than the total booking price (7500.00)**, which indicates it's **not the full payment**.